

Informe Técnico Integrador: Sistema de Semáforos Inteligentes

Asignatura: Paradigmas y Lenguajes de Programación (2026)

Integrantes: Dario Nahuel Gonzalez Machado

Tema: (Trabajo integrador) Sistema de semáforos inteligentes LISP (implementando funciones en Haskell)

NOTA IMPORTANTE: Este trabajo se ha realizado única y exclusivamente por mi. Ya que mis compañeros de equipo jamás se ponían de acuerdo entre ellos y no se hablaban, llegando a aplazar el trabajo y/o abandonar el cursado.

Lo realizaré solo ya que no me queda alternativa y tengo que obtener una buena nota. Gracias por su atención.

Introducción y Fundamentos del Diseño Funcional

El diseño de las funciones y el programa que se quiso lograr, fue hecho a partir del paradigma funcional, priorizando la **inmutabilidad** y la **pureza de las funciones**. Se evitó también el uso de **bucles imperativos** (solo se usó funciones de **orden superior** o de **recursividad de cola**) y se priorizo usar la documentación solicitada por las consignas del trabajo.

Pureza: La mayoría de las funciones implementadas (tanto en Lisp como en Haskell) son puras, lo que garantiza que ante una misma entrada, el sistema siempre produzca la misma salida, facilitando el testeo unitario.

Inmutabilidad: No se modifican estados globales (sólo funciones según requerimientos). En su lugar, se generan nuevos estados a partir de los anteriores mediante transformaciones (ej. **Transición**)

Declaratividad: El enfoque se centra en describir que es el semáforo (un conjunto de reglas de transición) en lugar de *cómo* debe cambiar el hardware paso a paso.

HASKELL INTRODUCCIÓN

- **Fundamento en Haskell:** A diferencia del enfoque imperativo, en Haskell cada función actúa como una transformación matemática. Esto elimina los efectos secundarios (como variables globales modificables), garantizando que, ante una misma entrada (tiempo), el sistema siempre retorna al mismo estado.
- **Declaratividad en Haskell:** Se utiliza **pattern matching** y **guards** para definir la lógica del semáforo, lo que permite que el código sea una representación directa de los requerimientos lógicos del problema.

Justificación de Herramientas y Librerías (Fase 2)

- **Quicklisp:** `quicklisp` (el cual necesito de uso de `sbcl` para poder hacerlo funcionar) lo utilice principalmente por el gestor de tiempos que posee, así como el uso de `local-time`, esencial para la generación de informes con “timestamps” precisos. Adaptando no sólo la hora sino la fecha exacta (aunque a veces sumamente larga si no la sabes manejar)
- **One-compiler:** se usó este gestor de lenguaje para poder usar ambos lenguajes (**LISP y HASKELL**) de forma más cómoda ya que no poseía el programa en mi computadora, el cual fue cómodo y me ayudó a corregir mis errores y ver de mejor forma mi código.

Bitácora de Depuración LISP

Necesidad de añadir una función residuo para sumar segundos extra a los semáforos... sino me daban porcentajes que no daban el 100% (por ejemplo cuando se pedía ingresar el porcentaje en 1 hora no retornaba el valor exacto)

```
;; =====  
;; FUNCIÓN: repartir-residuo  
;; NATURALEZA: Pura  
;; ESTRATEGIA: Funcion recursiva  
;; IMPACTO: No destructiva  
;; =====  
  
(defun repartir-residuo (residuo lista-semaforo)  
  (cond  
    ((null lista-semaforo) nil)  
    (t (cons (min (car lista-semaforo) residuo)  
              (repartir-residuo  
                (max 0 (- residuo (car lista-semaforo)))  
                (cdr lista-semaforo))))))
```

Error en porcentajes nuevamente, ya que no me daba el valor en flotante

```
(defun informe-distribucion [lista-semaforo]  
  (mapcar (lambda (x)  
            (format nil "~a%" (/ (* x 100) (duracion-ciclo lista-semaforo))))  
          lista-semaforo)  
)
```

salida:

```
NIL
("500/9%" "25/9%" "125/3%")
```

La solución fue ponerle formato ,2f y multiplicarlo por 100.0

```
(defun informe-distribucion (lista-semaforo)
  "Calcula los porcentajes de luces, usa formato 2f para que use solo dos decimales y 100.0 para transformarlo en flotante"
  (mapcar (lambda (x)
    (format nil "~,2F%" (/ (* x 100.0) (duracion-ciclo lista-semaforo))))
    lista-semaforo))
)
```

Error de sintaxis en reduce (orden superior)

Al parecer se tiene que usar # ' seguido del nombre de la función para que encuentre la función llamada así, sino tirara error

```
(defun duracion-ciclo (lista-semaforo)
  "Recibe una lista con la duracion de cada luz y calcula el tiempo total del ciclo"
  (reduce + lista-semaforo))
```

Output:

```
Unhandled UNDEFINED-FUNCTION in thread #<SB-THREAD:THREAD "main thread" RUNNING
                                     {10005184C3}>:
  The function COMMON-LISP:NIL is undefined.
```

Solución:

```
(reduce #' + lista-semaforo))
```

Otra función que fue algo problemática fue esta, que al olvidarme de ponerle "quote" quería ejecutarla como función. Además al arreglarlo me daba un dato que era erróneo por el hecho de ponerle "menor a"

```
(defun Mi-timer (tiempo)
  "La funcion position busca el primer valor que da true y nth obtiene el valor que esta en esa posicion"
  (nth (position t (mapcar (lambda (x) (< (mod tiempo 216) x))
    (120 126 216)))
    '(en-verde en-amarillo en-rojo)))
```

```
; compilation unit finished
; caught 1 ERROR condition
Unhandled SB-INT:COMPILED-PROGRAM-ERROR in thread #<SB-THREAD:THREAD "main thread" RUNNING
                                     {10005184C3}>:

  Execution of a form compiled with errors.
Form:
  (120 126 216)
Compile-time error:
  illegal function call
```

Solucion: agregue "menor o igual a" y puse quote

```
(defun Mi-timer (tiempo)
  "La funcion position busca el primer valor que da true y nth obtiene el valor que esta en esa posicion"
  (nth (position t (mapcar (lambda (x) (<= (mod tiempo 216) x))
    '(120 126 216)))
    '(en-verde en-amarillo en-rojo)))
```

Un error que me ocurrió al usar el sbcl es que no me generaba el txt en la computadora. Luego después de eso si me lo dejo implementar cuando cambie el código...

```
debugger invoked on a SB-C::INPUT-ERROR-IN-LOAD in thread
#<THREAD tid=2480 "main thread" RUNNING {1104030083}>:
  READ error during LOAD:

    end of file on #<SB-INT:FORM-TRACKING-STREAM for "file c:\\users\\thecrazy\\downloads\\helloworld.lsp" {11025EF4F3}>

    (in form starting at line: 218, column: 0, position: 8639)

Type HELP for debugger help, or (SB-EXT:EXIT) to exit from SBCL.

restarts (invokable by number or by possibly-abbreviated name):
  0: [ABORT] Abort loading file "c:\\users\\thecrazy\\downloads\\helloworld.lsp".
  1:      Exit debugger, returning to top level.

(SB-C:COMPILER-ERROR SB-C::INPUT-ERROR-IN-LOAD :CONDITION #<END-OF-FILE {1102A1BBA3}> :POSITION 8639 :LINE/COL (218 . 0)
c:\\users\\thecrazy\\downloads\\helloworld.lsp" {11025EF4F3}>))
0] 0

T
* (informe '(1717560000 1717865098 1717594300))
```

La solución fue cambiar el código de recursividad que intente y poner un mapcar dentro del código y hacer una función lambda usando un let para pasar el tiempo y los demás valores como dato local.

Función anterior:

```
;; =====
;; FUNCIÓN: leer-recursivo
;; NATURALEZA: Pura
;; ESTRATEGIA: Funcion recursiva
;; IMPACTO: No destructiva
;; =====

(defun leer-recursivo (stream datos)
  (when (and datos (cdr datos))
    (if (not (equal (Mi-timer (car datos))
      (Mi-timer (cadr datos))))
      (format stream "~a - Transición: ~a -> ~a~%"
        (hora-local)
        (Mi-timer (car datos))
        (Mi-timer (cadr datos)))
      (leer-recursivo stream (cdr datos))))

;; =====
;; FUNCIÓN: informe
;; NATURALEZA: Impura
;; ESTRATEGIA: Composición de funciones
;; IMPACTO: No destructiva
;; =====

(defun informe (datos)
  (with-open-file (stream "informe-ejecucion-semaforo.txt"
    :direction :output
    :if-exists :supersede
    :if-does-not-exist :create)
    (format stream "Informe de Ejecución del Sistema Semafórico~%")
    (format stream "=====~%")
    (leer-recursivo stream datos)
    (format stream "~%--- Fin del Informe ---~%")))
```

Solución:

```
(defun informe (datos)
  (with-open-file (stream "informe-ejecucion-semaforo.txt"
    :direction :output
    :if-exists :supersede
    :if-does-not-exist :create)
    (format stream "Informe de Ejecución del Sistema Semafórico~%")
    (format stream "=====~%")
    (mapcar #'(lambda (tiempo)
      (let* ((fecha-hoy (local-time:format-timestring nil
        (local-time:unix-to-timestamp tiempo)
        :format '(:year "-" :month "-" :day " " :hour ":" :min ":" :sec)))
        (color-actual (mi-timer tiempo))
        (color-anterior (cond
          ((equal (mi-timer tiempo) 'en-rojo) 'en-verde)
          ((equal (mi-timer tiempo) 'en-amarillo) 'en-rojo)
          ((equal (mi-timer tiempo) 'en-verde) 'en-amarillo))))
        (format stream "~a - Transición: ~a -> ~a~%"
          fecha-hoy
          color-anterior
          color-actual)))
      datos)
    (format stream "~%--- Fin del Informe ---~%")))
```

Bitácora de Depuración HASKELL

- **Gestión de tipos:** En Lisp, se superó la ambigüedad en el manejo de listas mediante funciones de orden superior (**mapcar**). En Haskell, el principal reto fue la concordancia en las funciones, resuelto mediante el uso de “WildCards” (**_**).
- **Manejo de I/O:** Se documentó la transición entre lógica pura (cálculo de tiempos) e impura (escritura en archivos o consola), utilizando **with-open-file** en Lisp y **IO()** en Haskell para encapsular los efectos secundarios.
- **Errores sintácticos:** Se registraron bugs comunes por balanceo de paréntesis en Lisp y errores de alcance en Haskell al definir funciones con nombres duplicados o que simplemente no me permitían hacer uso de las variables porque no tenían comillas como así, estaban mal tipeadas saltándose algún parentesis o intentando usar otra sintaxis como es la de LISP

Función utilizada:

```
main :: IO ()
main = print $ transicion EnVerde amarillo
```

Al hacer uso de esos comandos, me salió un error... y es que yo defino al segundo argumento como string y no como un dato.

```
Output:

Main.hs:29:35: error: Variable not in scope: amarillo :: String
      |
29 | main = print $ transicion EnVerde amarillo
      |                                   ^^^^^^^
```

Debería haber definido data Estado = verde | amarillo etc...para que funcione correctamente

```
data Color = EnVerde | EnAmarillo | EnRojo deriving (Show, Eq)
```

Ejes Comparativos (Fase 3: Lisp vs. Haskell)

Seguridad y sintaxis del tipo de dato

- **Lisp:** La flexibilidad permite prototipar rápido, sin embargo, requiere pruebas exhaustivas para asegurar que un valor inesperado no rompa la ejecución. Este método es más dinámico pero menos legible en mi opinión.
- **Haskell:** El uso de **ADT 's**. Un ADT es una forma de definir tipos de datos personalizados combinando otros tipos mediante operaciones de suma (alternativas) y producto (agrupaciones). En el caso del semáforo, utilizamos un tipo suma: **data Color = EnVerde | EnAmarillo | EnRojo**. Este tipo indica que un dato de tipo **Color** puede ser estrictamente una de esas tres opciones lo que obliga a definir exhaustivamente los casos posibles.

El compilador actúa como una herramienta de verificación lógica, impidiendo que el código compile si hay estados no cubiertos

La lógica del sistema intenta asignar un estado que no está definido en el **data Color** (por ejemplo, un hipotético '**en-azul**'), el compilador detectará la inconsistencia y **se negará a compilar el programa**.

Esto supera la seguridad de los lenguajes dinámicos como Lisp, donde un error de estado podría pasar desapercibido hasta que el programa se ejecute y falle en tiempo real.

Estructura de Control (Condicionales)

- **Lisp (Cond e if):** Utiliza estructuras de control basadas en evaluaciones de listas. La función **position** combinada con **nth** es un ejemplo de cómo Lisp mapea una lógica de selección sobre una estructura de datos.
- **Haskell (Guards y Pattern Matching):** Haskell utiliza **Guards (|)**, para evaluar condiciones booleanas y **Pattern Matching** para descomponer la estructura de los

datos. Esto resulta en una semántica más cercana a la definición matemática formal que las estructuras condicionales imperativas.

Evaluación y Entorno

- **Lisp:** Mejor interacción con el entorno, permitiendo probar por ej. **Sistema-auditoría** en tiempo real. En Lisp, puedes modificar una función, recopilarla y probarla en el mismo proceso mientras el sistema sigue funcionando.
- **Haskell:** Haskell no intenta calcular todos los estados futuros del semáforo en memoria al iniciar el programa ya que destaca por su sistema de tipos y **evaluación perezosa**. Esta última permite que el sistema maneje estructuras conceptualmente infinitas , solo calcula el valor específico del color cuando es estrictamente necesario para la ejecución inmediata. El sistema solo consume memoria para los eventos de tráfico que se requieren en el instante actual, permitiendo manejar simulaciones de tiempo continuo sin riesgo de desbordamiento de memoria, algo que en lenguajes impuros requeriría una gestión manual de iteradores o bucles finitos.

Gestión de infinitos: Es posible definir una estructura conceptualmente infinita, como una lista de tiempo `[0..]`, y procesarla de manera eficiente.

Un ejemplo sería:

```
tiemposInfinitos :: [Int]
tiemposInfinitos = [0..]

estadosInfinitos :: [Color]
estadosInfinitos = map miTimer tiemposInfinitos

main :: IO ()
main = print $ take 200 estadosInfinitos
```

salida:

e, EnVerde, EnVerde, EnVerde, EnAmarillo, EnAmarillo, EnAmarillo, EnAmarillo, EnAmarillo, EnAmarillo, EnRojo, EnRojo, EnRojo, EnRojo, EnRojo, EnRojo, EnRojo,

Áreas de aplicación e industria

Debido a su naturaleza robusta y su capacidad para manejar sistemas complejos con alta precisión, Haskell es frecuentemente adoptado en sectores donde el costo de un error es extremadamente alto:

- **Finanzas y Fintech:** Es el área donde más destaca. La precisión matemática de Haskell lo hace ideal para el modelado de riesgos, sistemas de trading de alta frecuencia y contratos inteligentes.

- **Sistemas de Seguridad y Criptografía:** Gracias a su sistema de tipos estricto, es la elección lógica para sistemas que requieren pruebas formales de seguridad.
- **Compiladores y Herramientas de Desarrollo:** Muchas de las herramientas más potentes del ecosistema de desarrollo moderno (como el propio compilador **GHC**) están construidas en Haskell.
- **Procesamiento de Datos y Análisis:** Su capacidad para manejar flujos infinitos y transformaciones de datos complejas lo hace muy eficaz en el procesamiento de grandes cantidades de datos

Empresas reconocidas que utilizan Haskell

- **Meta (Facebook):** Utiliza Haskell para gestionar sus sistemas de lucha contra el spam y para la infraestructura de servidores de anuncios (sistema sigma).
- **Standard Chartered:** Un banco global que utiliza Haskell para modelar instrumentos financieros y optimizar sus motores de cálculo de precios.
- **GitHub:** Utiliza Haskell para gestionar y procesar sus complejos sistemas de análisis de código y seguridad (como el semántico),
- **IOG (Input Output Global):** Responsables de la plataforma cardano, una de las blockchain más seguras del mundo, desarrollada íntegramente en Haskell para garantizar la corrección técnica del protocolo.

Conclusión Final:

La experiencia de implementar una solución de semáforos comparando Common Lisp y Haskell ha sido algo divertida, tanto por el hecho de poder obtener nuevos conceptos o modos de resolver un problema con diferentes códigos. Como así, la flexibilidad dinámica y el entorno REPL de Lisp me permitieron una experimentación rápida y una iteración veloz, siendo un lenguaje extremadamente permisivo y potente para modelar.

Por otro lado, Haskell me hizo mirar demasiadas veces el código, más incluso que LISP. Al final entendí que el compilador a pesar de poseer un paradigma un poco distinto, es un aliado bastante bueno. El uso de Tipos de Datos Algebraicos (ADT) me permitió definir estados del sistema imposibles de romper, garantizando que el semáforo nunca pudiera encontrarse en un estado no definido. La evaluación perezosa fue el descubrimiento más impactante tanto por su capacidad de procesar flujos infinitos de eventos de tráfico, sin necesidad de gestionar manualmente la memoria o los ciclos de vida de los procesos, como así su sintaxis, que simplificó enormemente la lógica del programa en algo menos abstracto para lenguaje humano. En conclusión, he aprendido que la elección del lenguaje no es simplemente una cuestión de sintaxis, sino una decisión estratégica. Mientras que Lisp me otorgó libertad de movimiento, Haskell me brindó una arquitectura robusta y segura. Esta comparativa ha fortalecido mi comprensión sobre cómo la estructura del lenguaje moldea la arquitectura del software que construí y lo pueda implementar en un futuro en otro proyecto.

Bibliografía Utilizada (Paradigma funcional, conceptos y sintaxis)

- Wikibooks, una enciclopedia con el modo de uso e introducción a HASKELL
enlace <https://en.wikibooks.org/wiki/Haskell>

Se usó para implementar funciones a partir de la información básica y su sintaxis

- Marlow, S. (Ed.) (2010) Haskell 2010 Reporte online

Enlace: <https://www.haskell.org/onlinereport/haskell2010/>

Usado para justificar la sintaxis de **guards**, **pattern matching** y el comportamiento del compilador

- Página oficial de Haskell con toda la documentación

Enlace: <https://www.haskell.org/documentation/>

Usado para investigar cómo funciona la evaluación perezosa

- Quicklisp.org Librería de common Lisp

Enlace: <https://www.quicklisp.org/beta/>

Usado en la implementación y para conocer como es el uso e instalación de timestamps para el semáforo

- Haskell Wiki Contribuidores (2026). Haskell en la industria

Enlace: https://wiki.haskell.org/Haskell_in_industry

Necesario para saber quienes usan haskell hoy en día